Bertnew.py

Explanation of the Code and Its Components

This Python script is a **job recommendation and skill gap analysis system**. It fetches job listings from **Google Jobs, Jooble, and Adzuna**, extracts required skills using **Natural Language Processing (NLP)**, compares them with the user's skills, and identifies **skill gaps**.

📌 Explanation of Key Imports

```
import json
import os
import requests
from collections import Counter
from transformers import pipeline
import spacy
```

1. **json** – Handles JSON data (for storing and loading job data).
2. **os** – Interacts with the file system (checking and managing files).
3. **requests** – Sends API requests to fetch job data from online job boards.
4. **collections.Counter** – Counts occurrences of skills across job listings.
5. **transformers.pipeline** – Uses **pre-trained NLP models** for Named Entity Recognition (NER) to extract skills.
6. **spacy** – Processes job descriptions and extracts relevant skills.

📌 API Configuration

```
GOOGLE_API_KEY = "AIzaSy..."
CUSTOM_SEARCH_ENGINE_ID = "3741929af..."
JOOBLE_API_URL = "https://jooble.org/api/..."
ADZUNA_APP_ID = "1fd3cc0a"
ADZUNA_APP_KEY = "5b1f7014cdef3..."
TEMP_JOB_FILE = "temp_jobs.json"
```

- **Google API & Custom Search Engine ID** → Used to fetch jobs from Google Search.
- **Jooble API** → Fetches jobs from Jooble's job database.
- **Adzuna API** → Fetches job postings from Adzuna.

- **TEMP_JOB_FILE** → A JSON file used to store previously fetched jobs to avoid redundant API calls.

📌 Skill Extraction Using NLP

```
nlp = spacy.load("en_core_web_sm")

skill_extractor = pipeline("ner", model="dbmdz/bert-large-cased-finetuned-conll03-english")

KNOWN_SKILLS = {"python", "javascript", "react", "css", "html", "django", "typescript", "graphql"}
```

- **spaCy model (en_core_web_sm)** extracts skill-related words from job descriptions.
- **transformers.pipeline (dbmdz/bert-large-cased-finetuned-conll03-english)** is a **pre-trained BERT model** used for **Named Entity Recognition (NER)** to detect programming languages, technologies, and soft skills.
- **KNOWN_SKILLS** → A predefined set of skills for filtering relevant ones.

📌 Fetching Jobs from Different APIs

1️⃣ Fetching Jobs from Google

```
def fetch_google_jobs(queries, location=""):
    """Fetch job listings using Google Custom Search API."""
    all_jobs = []
    for query in queries:
        try:
            params = {
                'key': GOOGLE_API_KEY,
                'cx': CUSTOM_SEARCH_ENGINE_ID,
                'q': f"{query} {location}",
                'num': 10
            }
            response = requests.get("https://www.googleapis.com/customsearch/v1", params=params)
            response.raise_for_status()
            search_results = response.json()
            jobs = [
                {
                    "title": item.get("title", "No Title"),
```

```
            "description": item.get("snippet", "No Description"),
            "link": item.get("link", ""),
            "source": "Google Search"
        }
        for item in search_results.get("items", [])
      ]
      all_jobs.extend(jobs)
    except requests.RequestException as e:
      print(f"Error fetching jobs for query '{query}': {e}")
  return all_jobs
```

- This function **queries Google's Custom Search API** to find job postings based on user queries.
- The results are stored in **all_jobs**, each containing title, description, link, and source.

2️⃣ Fetching Jobs from Jooble

```
def fetch_jooble_jobs():
  """Fetch job listings using Jooble API."""
  try:
    response = requests.post(JOOBLE_API_URL, json={"keywords": "software engineer"})
    response.raise_for_status()
    jobs = response.json().get("jobs", [])
    all_jobs = [
      {
        "title": job.get("title", "No Title"),
        "description": job.get("snippet", "No Description"),
        "link": job.get("url", ""),
        "source": "Jooble"
      }
      for job in jobs
    ]
    return all_jobs
  except requests.RequestException as e:
```

```
    print(f"Error fetching jobs from Jooble: {e}")
    return []
```

- Sends an API request to **Jooble** and extracts jobs based on keywords.

3️⃣ Fetching Jobs from Adzuna

```
def fetch_adzuna_jobs(role, location):
    """Fetch job listings using Adzuna API."""
    try:
        url = f"https://api.adzuna.com/v1/api/jobs/in/search/1"
        params = {
            "app_id": ADZUNA_APP_ID,
            "app_key": ADZUNA_APP_KEY,
            "results_per_page": 10,
            "what": role,
            "where": location
        }
        response = requests.get(url, params=params)
        response.raise_for_status()
        data = response.json()
        jobs = [
            {
                "title": job.get("title", "No Title"),
                "description": job.get("description", "No Description"),
                "link": job.get("redirect_url", ""),
                "source": "Adzuna"
            }
            for job in data.get("results", [])
        ]
        return jobs
    except requests.RequestException as e:
        print(f"Error fetching jobs from Adzuna: {e}")
```

return []

- Queries **Adzuna** for jobs matching the given role and location.
- Extracts title, description, link, and source.

📌 **Skill Extraction**

```
def extract_required_skills(description):
    """Extract skills from job descriptions using NER model and spaCy."""
    try:
        doc = nlp(description.lower())
        extracted_skills = [token.text for token in doc if token.pos_ in ["NOUN", "PROPN", "VERB"] and len(token.text) > 2]
        entities = skill_extractor(description)
        ner_skills = [entity['word'] for entity in entities if entity.get('entity_group') == "SKILL"]
        all_skills = list(set(extracted_skills + ner_skills))
        required_skills = [skill.lower() for skill in all_skills if skill.lower() in KNOWN_SKILLS]
        return required_skills
    except Exception as e:
        print(f"Error in skill extraction: {e}")
        return []
```

- Uses **spaCy and BERT** to identify skill-related terms in job descriptions.
- Compares them with **KNOWN_SKILLS** and returns **required skills** for the job.

📌 **Identifying Skill Gaps**

```
def find_skill_gaps(user_skills, job_skills):
    """Identify missing skills by comparing user skills with job requirements."""
    return list(set(job_skills) - set(user_skills.keys()))
```

- Compares the **user's skills** (input) with the **skills required** for jobs.
- Returns **missing skills** that the user needs to learn.

📌 **Job Storage and Retrieval**

```
def store_jobs_in_json(jobs, file_path):
```

```
    """Save job data to a JSON file."""
    with open(file_path, "w") as json_file:
        json.dump(jobs, json_file, indent=4)
```

- **Stores fetched jobs in temp_jobs.json** to avoid redundant API calls.

```
def load_jobs_from_json(file_path):
    """Load job data from a JSON file."""
    if os.path.exists(file_path):
        with open(file_path, "r") as json_file:
            return json.load(json_file)
    return []
```

- **Loads jobs from JSON** instead of fetching them again from APIs.

📌 Running the Program

```
if __name__ == "__main__":
    main()
```

- Ensures that the script runs **only when executed as a standalone script**.

Summary

✅ **Fetches jobs from Google, Jooble, and Adzuna**
✅ **Extracts skills from job descriptions** using **spaCy** and **BERT-based NER models**
✅ **Compares user skills with job requirements** and finds skill gaps
✅ **Saves and loads job data from JSON to avoid redundant API calls**

Would you like a **refined version** or **additional features**? 😊

Details.py

Explanation of the Code and Its Components

This Python script is a **Job Recommendation System** that: ✅ **Fetches job postings** from **Google and Jooble APIs**
✅ **Extracts required skills** from job descriptions using **spaCy NLP**
✅ **Compares job skills with user skills** and finds **skill gaps**
✅ **Uses a GUI (Tkinter) to collect user details**
✅ **Recommends jobs based on user skills and job relevance**

📌 Explanation of Key Imports

```
import requests
import spacy
import time
import tkinter as tk
from tkinter import simpledialog
```

1. **requests** → Sends API requests to fetch job postings from Google and Jooble.
2. **spacy** → Uses **Natural Language Processing (NLP)** to extract required skills from job descriptions.
3. **time** → Introduces **delays between API retries** to avoid being blocked.
4. **tkinter** → Creates a **GUI pop-up window** to collect user details.
5. **simpledialog** → Allows **input dialogs** for user data collection.

📌 **API Configuration**

```
GOOGLE_API_KEY = "AIzaSy..."
CUSTOM_SEARCH_ENGINE_ID = "3741929af..."
JOOBLE_API_URL = "https://jooble.org/api/..."
MAX_RETRIES = 3  # Maximum number of retries for failed API requests
```

- **Google API & Custom Search Engine ID** → Used to **search job postings** using Google.
- **Jooble API** → Fetches **job listings** from Jooble.
- **MAX_RETRIES = 3** → Limits API request retries to **prevent excessive requests**.

📌 **Fetching Jobs from Google Custom Search API**

```
def fetch_google_jobs(queries=None, job_role=None, location=None):
    if queries is None:
        queries = [
            f"{job_role} jobs",
            "software engineer jobs",
            "python developer jobs",
            "remote software jobs"
        ]
```

```
if location:
    queries = [query + f" in {location}" for query in queries]

all_jobs = []
for query in queries:
    retries = 0
    while retries < MAX_RETRIES:
        try:
            params = {
                'key': GOOGLE_API_KEY,
                'cx': CUSTOM_SEARCH_ENGINE_ID,
                'q': query,
                'num': 10  # Limit to 10 results per query
            }
            response = requests.get("https://www.googleapis.com/customsearch/v1", params=params)
            response.raise_for_status()
            search_results = response.json()

            jobs = [({
                "title": item.get("title", "No Title"),
                "description": item.get("snippet", "No Description"),
                "link": item.get("link", ""),
                "source": "Google Search",
                "salary": item.get("formattedValue", ""),
                "company": item.get("displayLink", ""),
                "location": item.get("pagemap", {}).get("postalAddress", [{}])[0].get("addressLocality", "")
            }) for item in search_results.get("items", [])]

            all_jobs.extend(jobs)
            break  # Exit retry loop if successful
```

```
        except requests.RequestException as e:
            print(f"Error fetching jobs for query '{query}': {e}")
            retries += 1
            if retries < MAX_RETRIES:
                print(f"Retrying... ({retries}/{MAX_RETRIES})")
                time.sleep(2)  # Wait before retrying

    return all_jobs
```

✅ **Searches for jobs using Google's Custom Search API**
✅ **Retrieves job title, description, salary, company, and location**
✅ **Implements a retry mechanism** to **handle API failures**

📌 **Fetching Jobs from Jooble API**

```
def fetch_jooble_jobs(queries=None, job_role=None, location=None):
    if queries is None:
        queries = [
            f"{job_role}",
            "software engineer",
            "python developer",
            "remote software jobs"
        ]

    all_jobs = []
    for query in queries:
        retries = 0
        while retries < MAX_RETRIES:
            try:
                payload = {
                    'keywords': query,
                    'location': location if location else '',  # Use location if provided
                    'page': '1'
```

```
            }

            response = requests.post(JOOBLE_API_URL, json=payload)
            response.raise_for_status()
            jooble_results = response.json()

            jobs = [({
                "title": job.get("title", "No Title"),
                "description": job.get("snippet", "No Description"),
                "link": job.get("link", ""),
                "source": "Jooble",
                "salary": job.get("salary", ""),
                "company": job.get("company", ""),
                "location": job.get("location", "")
            }) for job in jooble_results.get("jobs", [])]

            all_jobs.extend(jobs)
            break  # Exit retry loop if successful

        except requests.RequestException as e:
            print(f"Error fetching jobs from Jooble for query '{query}': {e}")
            retries += 1
            if retries < MAX_RETRIES:
                print(f"Retrying... ({retries}/{MAX_RETRIES})")
                time.sleep(2)  # Wait before retrying

    return all_jobs
```

✅ **Fetches jobs from Jooble API**
✅ **Extracts job details including salary and location**
✅ **Handles API request failures with retry mechanism**

📌 **Extracting Required Skills from Job Descriptions**

```
def extract_required_skills(description):
    doc = nlp(description.lower())
    required_skills = [token.text for token in doc if token.pos_ in ["NOUN", "PROPN", "VERB"] and len(token.text) > 2]
    return required_skills
```

✅ **Uses spaCy NLP** to extract **technical skills** from job descriptions
✅ **Identifies relevant nouns, proper nouns, and verbs**

📌 **Matching Jobs Based on Role and Skills**

```
def enhance_role_matching(user_role, job_title, job_description):
    user_role_lower = user_role.lower()
    job_title_lower = job_title.lower()
    job_description_lower = job_description.lower()
    return user_role_lower in job_title_lower or user_role_lower in job_description_lower
```

✅ **Ensures job title or description matches user role**
✅ **Improves job recommendation relevance**

📌 **Recommending Jobs Based on Skill Matching**

```
def recommend_jobs(user_skills, job_data, user_role, min_skill_match=1):
    recommended_jobs = []
    skills = [skill.strip().lower() for skill in user_skills.split(',')]

    for job in job_data:
        job_description = job.get("description", "").lower()
        job_title = job.get("title", "").lower()

        if enhance_role_matching(user_role, job_title, job_description):
            job_required_skills = extract_required_skills(job_description)
            skill_matches = [skill for skill in skills if skill in job_required_skills]
            skill_gaps = [skill for skill in job_required_skills if skill not in skills]
```

```
        job_relevance_score = len(skill_matches)

        if job_relevance_score >= min_skill_match:
            job['skill_match_count'] = job_relevance_score
            job['skill_matches'] = skill_matches
            job['skill_gaps'] = skill_gaps
            job['relevance_score'] = job_relevance_score
            recommended_jobs.append(job)

    recommended_jobs.sort(key=lambda x: x.get('relevance_score', 0), reverse=True)
    return recommended_jobs
```

✅ **Compares job requirements with user skills**
✅ **Sorts jobs based on the number of matched skills**

📌 **Collecting User Details Using Tkinter**

```
def collect_user_details():
    root = tk.Tk()
    root.withdraw()

    name = simpledialog.askstring("User Details", "Name:")
    email = simpledialog.askstring("User Details", "Email:")
    job_role = simpledialog.askstring("User Details", "Job Role Applied For:")
    location = simpledialog.askstring("User Details", "Preferred Location:")
    skills = simpledialog.askstring("User Details", "Technical Skills (comma-separated):")

    return {
        "name": name,
        "email": email,
        "job_role": job_role,
        "location": location,
        "skills": skills
```

}

✅ **Creates a pop-up window to collect user details**
✅ **Stores data in a dictionary for later use**

📌 **Running the Job Recommendation System**

```
if __name__ == "__main__":
    main()
```

✅ **Runs the main function when executed as a script**

🚀 **Summary**

✅ **Fetches jobs from Google & Jooble**
✅ **Extracts required skills using NLP**
✅ **Matches jobs based on user skills**
✅ **Provides job recommendations & highlights skill gaps**

Would you like to add **LinkedIn job search** or **email notifications**? 😊

Job.py

📌 **Explanation of the Job Recommendation System Code**

This Python script is a **Job Recommendation System** that: ✅ **Fetches job postings** from **Google and Jooble APIs**
✅ **Extracts required skills** from job descriptions using **spaCy NLP**
✅ **Matches jobs based on user skills** and **highlights skill gaps**
✅ **Sorts jobs by relevance** (number of matched skills)

📌 **Explanation of Key Imports**

```
import requests
import spacy
```

1. **requests** → Sends API requests to **Google Custom Search** and **Jooble API**.
2. **spacy** → Uses **Natural Language Processing (NLP)** to extract **skills from job descriptions**.

📌 **API Configuration**

```
GOOGLE_API_KEY = "AIzaSy..."
CUSTOM_SEARCH_ENGINE_ID = "3741929af..."
JOOBLE_API_URL = "https://jooble.org/api/..."
```

- **Google API & Custom Search Engine ID** → Used to **search job postings** using Google.
- **Jooble API** → Fetches **job listings** from Jooble.

📌 **Fetching Jobs from Google Custom Search API**

```
def fetch_google_jobs(queries=None):
    if queries is None:
        queries = [
            "software engineer jobs",
            "python developer jobs",
            "remote software jobs",
            "entry level programming jobs"
        ]

    all_jobs = []
    for query in queries:
        try:
            params = {
                'key': GOOGLE_API_KEY,
                'cx': CUSTOM_SEARCH_ENGINE_ID,
                'q': query,
                'num': 10  # Limit to 10 results per query
            }
            response = requests.get("https://www.googleapis.com/customsearch/v1", params=params)
            response.raise_for_status()
            search_results = response.json()

            jobs = [ {
                "title": item.get("title", "No Title"),
                "description": item.get("snippet", "No Description"),
                "link": item.get("link", ""),
                "source": "Google Search"
```

```
        } for item in search_results.get("items", [])]

        all_jobs.extend(jobs)

    except requests.RequestException as e:
        print(f"Error fetching jobs for query '{query}': {e}")

return all_jobs
```

✅ **Searches for jobs using Google's Custom Search API**
✅ **Retrieves job title, description, and link**
✅ **Handles API request failures gracefully**

📌 **Fetching Jobs from Jooble API**

```
def fetch_jooble_jobs(queries=None):
    if queries is None:
        queries = [
            "software engineer",
            "python developer",
            "remote software jobs",
            "entry level programming"
        ]

    all_jobs = []
    for query in queries:
        try:
            payload = {
                'keywords': query,
                'location': '',  # Empty location for global search or set to a specific location
                'page': '1'
            }
```

```
            response = requests.post(JOOBLE_API_URL, json=payload)
            response.raise_for_status()
            jooble_results = response.json()

            jobs = [ {
                "title": job.get("title", "No Title"),
                "description": job.get("snippet", "No Description"),
                "link": job.get("link", ""),
                "source": "Jooble"
            } for job in jooble_results.get("jobs", [])]

            all_jobs.extend(jobs)

        except requests.RequestException as e:
            print(f"Error fetching jobs from Jooble for query '{query}': {e}")

    return all_jobs
```

✅ **Fetches jobs from Jooble API**
✅ **Extracts job details including title, description, and link**
✅ **Handles API request failures safely**

📌 **Extracting Required Skills from Job Descriptions**

```
def extract_required_skills(description):
    # Process the job description using spaCy
    doc = nlp(description.lower())
    # Extracting potential skills by filtering for nouns, proper nouns, and verbs
    required_skills = [token.text for token in doc if token.pos_ in ["NOUN", "PROPN", "VERB"] and len(token.text) > 2]
    return required_skills
```

✅ **Uses spaCy NLP** to extract **technical skills** from job descriptions
✅ **Identifies relevant nouns, proper nouns, and verbs**

📌 **Recommending Jobs Based on Skill Matching**

```
def recommend_jobs(user_skills, job_data):
    recommended_jobs = []
    skills = [skill.strip().lower() for skill in user_skills.split(',')]

    for job in job_data:
        job_description = job.get("description", "").lower()
        job_title = job.get("title", "").lower()

        # Extract the required skills from job description
        job_required_skills = extract_required_skills(job_description)

        # Match user's skills with required skills
        skill_matches = [skill for skill in skills if skill in job_required_skills]
        skill_gaps = [skill for skill in job_required_skills if skill not in skills]

        if skill_matches:
            job['skill_match_count'] = len(skill_matches)
            job['skill_matches'] = skill_matches
            job['skill_gaps'] = skill_gaps
            recommended_jobs.append(job)

    # Sort jobs by the number of skill matches (desc)
    recommended_jobs.sort(key=lambda x: x.get('skill_match_count', 0), reverse=True)

    return recommended_jobs
```

✅ **Compares job requirements with user skills**
✅ **Sorts jobs based on the number of matched skills**

📌 **Running the Job Recommendation System**

```
def main():
```

```
    try:
        # Fetch job data from Google and Jooble
        google_jobs = fetch_google_jobs()
        jooble_jobs = fetch_jooble_jobs()

        # Combine job data from both sources
        job_data = google_jobs + jooble_jobs

        print(f"Total jobs fetched: {len(job_data)}")

        # Get user skills
        user_skills = input("Enter your skills (comma-separated): ").strip()

        # Get recommended jobs
        recommended_jobs = recommend_jobs(user_skills, job_data)

        print("\nRecommended Jobs:")
        for job in recommended_jobs[:10]:  # Limit to top 10 recommendations
            print(f"Title: {job.get('title')}")
            print(f"Description: {job.get('description')[:300]}...")
            print(f"Skill Matches: {job.get('skill_match_count', 0)}")
            print(f"Matched Skills: {', '.join(job.get('skill_matches', []))}")
            print(f"Skill Gaps: {', '.join(job.get('skill_gaps', []))}")
            print(f"Source: {job.get('source')}")
            print(f"Link: {job.get('link')}")
            print("---")

    except Exception as e:
        print(f"An error occurred: {e}")

# Run the main function
```

```
if __name__ == "__main__":
    main()
```

✅ **Fetches jobs from Google & Jooble**
✅ **Extracts required skills using NLP**
✅ **Matches jobs based on user skills**
✅ **Provides job recommendations & highlights skill gaps**

🚀 **Summary**

- 🔹 **Fetches job listings** from **Google & Jooble**
- 🔹 **Uses NLP (spaCy)** to extract required skills
- 🔹 **Compares job skills with user skills**
- 🔹 **Highlights skill gaps**
- 🔹 **Ranks jobs by skill match**

Would you like to add **LinkedIn job search** or **email notifications**? 😊

📌 **Explanation of the Job Matching System Code**

This Python script is an **AI-powered job matching system** that:
✅ **Fetches jobs** from **Google API**
✅ **Extracts required skills** from job descriptions using **NLP**
✅ **Finds skill gaps** & **recommends certification courses**
✅ **Uses Machine Learning (ML) techniques** for **clustering & categorization**

📌 **Explanation of Key Imports**

```
import json
import os
from transformers import pipeline
import requests
import spacy
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import MiniBatchKMeans
from sklearn.naive_bayes import MultinomialNB
from sklearn.preprocessing import LabelEncoder
import numpy as np
```

- **json & os** → Handle **job data storage in JSON**.
- **requests** → Fetch **job listings from APIs**.

- **spacy** → Extract **skills from job descriptions** using **NLP**.
- **transformers.pipeline** → Uses **NER (Named Entity Recognition) model** to find **relevant skills**.
- **sklearn** → Implements **Machine Learning (ML) techniques**:
    - **TF-IDF Vectorizer** → Converts job descriptions into **numerical features**.
    - **MiniBatchKMeans** → Clusters **similar jobs together**.
    - **Naïve Bayes Classifier** → Categorizes **jobs into predefined labels**.
    - **LabelEncoder** → Encodes categorical **job categories into numbers**.
- **numpy** → Works with **numerical data & arrays**.

📌 **API Configuration**

```
GOOGLE_API_KEY = "AIzaSy..."
CUSTOM_SEARCH_ENGINE_ID = "3741929af..."
TEMP_JOB_FILE = "fetched_jobs.json"  # Stores job listings temporarily
```

- **Google API & Custom Search Engine ID** → Used for **job searches**.
- **TEMP_JOB_FILE** → Stores **cached jobs** to **avoid redundant API calls**.

📌 **Fetching Jobs from Google**

```
def fetch_google_jobs(queries, location=""):
    all_jobs = []
    for query in queries:
        try:
            params = {
                'key': GOOGLE_API_KEY,
                'cx': CUSTOM_SEARCH_ENGINE_ID,
                'q': f"{query} {location}",
                'num': 10  # Fetch 10 results
            }
            response = requests.get("https://www.googleapis.com/customsearch/v1", params=params)
            response.raise_for_status()
            search_results = response.json()
```

```
            jobs = [dict(title=item.get("title", "No Title"),
                         description=item.get("snippet", "No Description"),
                         link=item.get("link", ""),
                         source="Google Search")
                    for item in search_results.get("items", [])]
            all_jobs.extend(jobs)

        except requests.RequestException as e:
            print(f"Error fetching jobs for query '{query}': {e}")

    return all_jobs
```

✅ **Sends an API request to Google**
✅ **Extracts job titles, descriptions, and links**
✅ **Handles API request failures**

📌 **Extracting Required Skills from Job Descriptions**

```
# Load NLP model
nlp = spacy.load("en_core_web_sm")
skill_extractor = pipeline("ner", model="dbmdz/bert-large-cased-finetuned-conll03-english")

def extract_required_skills(description):
    try:
        entities = skill_extractor(description)
        required_skills = [entity['word'] for entity in entities if entity.get('entity_group') == "SKILL"]
        return required_skills
    except Exception as e:
        print(f"Error in skill extraction: {e}")
        return []
```

✅ **Uses NLP & Transformers** to extract **skills**
✅ **Finds relevant skills** from **job descriptions**
✅ **Handles exceptions for robustness**

📌 **Identifying Skill Gaps**

```
def find_skill_gaps(user_skills, job_skills):
    skill_gaps = {}
    for job, skills in job_skills.items():
        gaps = list(set(skills) - set(user_skills.keys()))
        if gaps:
            skill_gaps[job] = gaps
    return skill_gaps
```

✅ **Compares user skills with job requirements**
✅ **Finds missing skills** to **suggest learning paths**

📌 **Fetching Certification Courses for Missing Skills**

```
def fetch_certification_courses(missing_skills):
    courses = {}
    for skill in missing_skills:
        try:
            params = {
                'key': GOOGLE_API_KEY,
                'cx': CUSTOM_SEARCH_ENGINE_ID,
                'q': f"{skill} certification course",
                'num': 5
            }
            response = requests.get("https://www.googleapis.com/customsearch/v1", params=params)
            response.raise_for_status()
            search_results = response.json()

            courses[skill] = [dict(title=item.get("title", "No Title"),
                                   link=item.get("link", ""))
                              for item in search_results.get("items", [])]

        except requests.RequestException as e:
```

```
            print(f"Error fetching courses for skill '{skill}': {e}")
            courses[skill] = []

    return courses
```

✅ **Fetches certification courses** for **missing skills**
✅ **Uses Google API** to recommend **relevant learning resources**

📌 **Clustering Jobs Using MiniBatch K-Means**

```
def cluster_job_descriptions(jobs, n_clusters=5):
    descriptions = [job['description'] for job in jobs]
    vectorizer = TfidfVectorizer(stop_words='english')
    tfidf_matrix = vectorizer.fit_transform(descriptions)

    kmeans = MiniBatchKMeans(n_clusters=n_clusters, random_state=42)
    clusters = kmeans.fit_predict(tfidf_matrix)

    for i, job in enumerate(jobs):
        job['cluster'] = int(clusters[i])

    return jobs, kmeans
```

✅ **Groups similar jobs into clusters**
✅ **Uses TF-IDF & K-Means for clustering**

📌 **Categorizing Jobs Using Naïve Bayes**

```
def categorize_jobs(jobs, categories):
    descriptions = [job['description'] for job in jobs]
    labels = np.random.choice(categories, len(descriptions))  # Random labels for demonstration

    vectorizer = TfidfVectorizer(stop_words='english')
    tfidf_matrix = vectorizer.fit_transform(descriptions)
    label_encoder = LabelEncoder()
```

```
    encoded_labels = label_encoder.fit_transform(labels)

    model = MultinomialNB()
    model.fit(tfidf_matrix, encoded_labels)

    predictions = model.predict(tfidf_matrix)
    for i, job in enumerate(jobs):
        job['category'] = label_encoder.inverse_transform([predictions[i]])[0]

    return jobs
```

✅ **Classifies jobs into predefined categories**
✅ **Uses TF-IDF & Naïve Bayes for classification**

📌 **Main Execution: Running the Job Matching System**

```
def main():
    try:
        location = input("Enter your preferred location: ").strip()
        preferred_role = input("Enter your preferred job role: ").strip()
        user_skill_ratings = get_user_skill_ratings()

        if not user_skill_ratings:
            print("Failed to parse skills and ratings. Exiting.")
            return

        # Load or fetch jobs
        jobs = load_jobs_from_json(TEMP_JOB_FILE)
        if not jobs:
            queries = [preferred_role, "software engineer", "python developer", "remote jobs"]
            jobs = fetch_google_jobs(queries, location)
            store_jobs_in_json(jobs, TEMP_JOB_FILE)
```

```
        print(f"Total jobs loaded: {len(jobs)}")

        # Cluster jobs
        jobs, kmeans_model = cluster_job_descriptions(jobs)
        print(f"Jobs clustered into {kmeans_model.n_clusters} groups.")

        # Categorize jobs
        categories = ["Software", "Management", "Marketing", "Finance", "Healthcare"]
        jobs = categorize_jobs(jobs, categories)

        # Skill gaps
        job_skills = {job['title']: extract_required_skills(job.get("description", "")) for job in jobs}
        skill_gaps = find_skill_gaps(user_skill_ratings, job_skills)

        # Recommend certifications
        missing_skills = set(skill for gaps in skill_gaps.values() for skill in gaps)
        certification_courses = fetch_certification_courses(missing_skills)

    except Exception as e:
        print(f"An error occurred: {e}")

if __name__ == "__main__":
    main()
```

🚀 **This job recommendation system:**
✅ **Fetches jobs**
✅ **Extracts skills**
✅ **Finds skill gaps**
✅ **Suggests courses**
✅ **Clusters & categorizes jobs using ML**

Would you like to **optimize search queries** or **add resume matching**? 😊